

Experiment No. 4

Student Name: Ojaswini Thote

PRN: 24070521048

File Path: file:///Users/ojaswinithote/Desktop/Javascript/EXP%204/1/index.html |
file:///Users/ojaswinithote/Desktop/Javascript/EXP%204/2/index1.html

Experiment Title

**Implementation of Function Types, Scope, Closures and Exception Handling using Try-Catch;
Development of a Palindrome Checker and Vehicle Registration Validator**

Software / Tools Required

- Visual Studio Code
- Google Chrome
- HTML5
- CSS3
- JavaScript (ES6)

Experiment Program Code

Task 4.a — Palindrome Checker (`index.html` + `script.js`)

`Task4.a/index.html`

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Palindrome Checker</title>

  <style>
    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      min-height: 100vh;
      font-family: Arial, sans-serif;

      background: linear-gradient(135deg, #667eea, #764ba2, #ff758c);

      display: flex;
      justify-content: center;
```

```
        align-items: center;
    }

    .container {
        width: 400px;
        padding: 40px;

        background-color: white;
        border-radius: 20px;

        text-align: center;

        box-shadow: 0 10px 30px rgba(0, 0, 0, 0.25);
    }

    h1 {
        margin-top: 0;
        margin-bottom: 10px;

        color: #6c5ce7;
        font-size: 30px;
    }

    .subtitle {
        color: #777;
        font-size: 14px;
        margin-bottom: 30px;
    }

    label {
        display: block;
        margin-bottom: 10px;

        color: #444;
        font-size: 17px;
        font-weight: bold;
    }

    #word {
        width: 100%;
        padding: 13px;

        font-size: 16px;

        border: 2px solid #ddd;
        border-radius: 10px;
        outline: none;

        transition: 0.3s;
    }

    #word:focus {
        border-color: #6c5ce7;
        box-shadow: 0 0 8px rgba(108, 92, 231, 0.3);
    }

    button {
        width: 100%;
        padding: 13px;
        margin-top: 20px;

        background: linear-gradient(90deg, #6c5ce7, #fd79a8);

        color: white;
    }
}
```

```

        font-size: 16px;
        font-weight: bold;

        border: none;
        border-radius: 10px;

        cursor: pointer;
        transition: 0.3s;
    }

    button:hover {
        transform: translateY(-2px);
        box-shadow: 0 5px 15px rgba(108, 92, 231, 0.4);
    }

    #result {
        min-height: 22px;
        margin-top: 25px;

        color: #6c5ce7;
        font-size: 18px;
        font-weight: bold;
    }
</style>
</head>

<body>

    <div class="container">

        <h1>Palindrome Checker</h1>

        <p class="subtitle">
            Enter a word and check if it reads the same backwards.
        </p>

        <label for="word">Enter a word:</label>

        <input
            type="text"
            id="word"

        >

        <button type="button" onclick="checkPalindrome()">
            Check Palindrome
        </button>

        <p id="result"></p>

    </div>

    <script src="script.js"></script>

</body>

</html>

```

`Task4.a/script.js``

```

// Function Declaration
function reverseString(str) {
    return str.split("").reverse().join("");
}

```

```

// Closure Example
function message() {

    var msg = "Palindrome Checker";

    function display() {
        return msg;
    }

    return display;
}

// Palindrome Checker
function checkPalindrome() {

    try {

        // Get input
        var word = document.getElementById("word").value.trim();

        // Check empty input
        if (word === "") {
            throw "Please enter a word.";
        }

        // Check only alphabets
        if (!/^[a-zA-Z]+$/.test(word)) {
            throw "Only alphabets are allowed.";
        }

        // Minimum 3 characters
        if (word.length < 3) {
            throw "Word must contain at least 3 alphabets.";
        }

        // Maximum 20 characters
        if (word.length > 20) {
            throw "Word must contain maximum 20 alphabets.";
        }

        // Convert input to lowercase
        let input = word.toLowerCase();

        // Reverse the input
        var reversed = reverseString(input);

        // Check palindrome
        if (input === reversed) {

            document.getElementById("result").innerHTML =
                "✓ " + word + " is a Palindrome!";

        } else {

```

```

        document.getElementById("result").innerHTML =
            "X " + word + " is not a Palindrome!";
    }

}

catch (error) {

    // Show validation warning as popup
    alert(error);

}

}

```

The Palindrome Checker accepts a word or phrase from the user and determines whether it reads the same forwards and backwards.

Features:

- User input for words or phrases
- Removes spaces and special characters before checking
- Case-insensitive comparison
- Displays whether the input is a palindrome
- Uses exception handling for empty input
- Demonstrates function declaration, function expression, arrow function, scope, and closures

The application uses JavaScript functions to clean the input, reverse the string, compare values, and display the result dynamically. It also implements a closure to demonstrate variable persistence and a try-catch block for error handling.

Task 4.b — Vehicle Registration Validator (`index1.html` + `script1.js`)

`Task4.b/index1.html`

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Vehicle Registration Validator</title>

  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      min-height: 100vh;
      font-family: Arial, sans-serif;
      background-color: #fce7f3;

      display: flex;
      justify-content: center;

```

```

        align-items: center;
    }

    .container {
        width: 450px;
        background-color: white;

        border-radius: 18px;
        overflow: hidden;

        border: 1px solid #f5c2d7;

        box-shadow: 0 12px 35px rgba(157, 70, 107, 0.18);
    }

    .header {
        background-color: #be6f8f;
        color: white;

        text-align: center;
        padding: 30px 20px;
    }

    .icon {
        width: 60px;
        height: 60px;

        margin: 0 auto 15px;

        background-color: #fff0f6;

        border-radius: 50%;

        display: flex;
        justify-content: center;
        align-items: center;

        font-size: 30px;
    }

    .header h2 {
        font-size: 25px;
        margin-bottom: 7px;
    }

    .header p {
        font-size: 14px;
        color: #fff0f6;
    }

    .form-area {
        padding: 35px;
    }

    label {
        display: block;

        font-size: 15px;
        font-weight: bold;

        color: #713c52;

        margin-bottom: 9px;
    }

```

```

input {
  width: 100%;

  padding: 14px 15px;

  border: 2px solid #f0c6d7;
  border-radius: 9px;

  font-size: 17px;
  letter-spacing: 2px;

  background-color: #fffafb;

  outline: none;

  transition: 0.25s;
}

input:focus {
  border-color: #be6f8f;

  box-shadow: 0 0 0 4px rgba(190, 111, 143, 0.12);
}

input::placeholder {
  color: #b99aa7;
}

.example {
  margin-top: 8px;

  color: #956c7d;
  font-size: 13px;
}

button {
  width: 100%;

  margin-top: 25px;
  padding: 14px;

  background-color: #be6f8f;
  color: white;

  border: none;
  border-radius: 9px;

  font-size: 16px;
  font-weight: bold;

  cursor: pointer;

  transition: 0.25s;
}

button:hover {
  background-color: #a95779;

  transform: translateY(-2px);

  box-shadow: 0 6px 15px rgba(169, 87, 121, 0.25);
}

```

```
#result {
    margin-top: 22px;

    padding: 13px;

    border-radius: 8px;

    font-size: 15px;
    font-weight: bold;

    text-align: center;
}

#result:empty {
    display: none;
}

.valid {
    background-color: #e8f7ed;
    color: #287a47;

    border: 1px solid #a8dbb8;
}

.invalid {
    background-color: #fde8ed;
    color: #b23a57;

    border: 1px solid #f2adb8;
}

.format-box {
    margin-top: 25px;

    background-color: #fff5f8;

    border: 1px solid #f3cfd8;
    border-radius: 9px;

    padding: 15px;

    text-align: center;
}

.format-box p {
    color: #956c7d;
    font-size: 13px;

    margin-bottom: 6px;
}

.format-box strong {
    color: #9d466b;

    letter-spacing: 2px;
    font-size: 17px;
}
</style>

</head>

<body>

    <div class="container">
```



```

<div class="header">

    <div class="icon">
        □
    </div>

    <h2>Vehicle Registration Validator</h2>

    <p>Check whether a vehicle registration number is valid</p>
</div>

<div class="form-area">

    <label for="regNumber">
        Registration Number
    </label>

    <input
        type="text"
        id="regNumber"
        placeholder="MH12AB1234"
        maxlength="10"
    >

    <p class="example">
        Enter the registration number without spaces.
    </p>

    <button type="button" onclick="validateRegistration()">
        Validate Registration
    </button>

    <p id="result"></p>

    <div class="format-box">

        <p>Required Format</p>

        <strong>AA 00 AA 0000</strong>

    </div>

</div>

</div>

<!-- JAVASCRIPT -->

<script>

    function validateRegistration() {

        const regNumber =
            document.getElementById("regNumber").value.trim();

        const result =
            document.getElementById("result");

```

```

try {

    // Check empty input
    if (regNumber === "") {
        throw new Error(
            "Registration number cannot be empty."
        );
    }

    // Check length
    if (regNumber.length !== 10) {
        throw new Error(
            "Registration number must be exactly 10 characters."
        );
    }

    // First 2 characters = uppercase letters
    if (!/^[A-Z]{2}$/.test(regNumber.substring(0, 2))) {

        throw new Error(
            "First 2 characters must be uppercase alphabets."
        );
    }

    // Characters 3 and 4 = digits
    if (!/^[0-9]{2}$/.test(regNumber.substring(2, 4))) {

        throw new Error(
            "Characters 3 and 4 must be digits."
        );
    }

    // Characters 5 and 6 = uppercase letters
    if (!/^[A-Z]{2}$/.test(regNumber.substring(4, 6))) {

        throw new Error(
            "Characters 5 and 6 must be uppercase alphabets."
        );
    }

    // Last 4 characters = digits
    if (!/^[0-9]{4}$/.test(regNumber.substring(6, 10))) {

        throw new Error(
            "Last 4 characters must be digits."
        );
    }

    // VALID
    result.textContent =
        "Valid Registration Number";

    result.className = "valid";

} catch (error) {

```

```

        // INVALID
        result.textContent =
            "Invalid: " + error.message;

        result.className = "invalid";
    }
}

</script>

</body>

</html>

```

Task4.b/script1.js`

```

function validateRegistration() {

    const regNumber = document.getElementById("regNumber").value.trim();
    const result = document.getElementById("result");

    try {
        if (regNumber === "") {
            throw new Error("Registration number cannot be empty.");
        }

        if (regNumber.length !== 10) {
            throw new Error("Registration number must be exactly 10 characters.");
        }

        if (!/^[A-Z]{2}$/.test(regNumber.substring(0, 2))) {
            throw new Error("First 2 characters must be uppercase alphabets.");
        }

        if (!/^[0-9]{2}$/.test(regNumber.substring(2, 4))) {
            throw new Error("Characters 3 and 4 must be digits.");
        }

        if (!/^[A-Z]{2}$/.test(regNumber.substring(4, 6))) {
            throw new Error("Characters 5 and 6 must be uppercase alphabets.");
        }

        if (!/^[0-9]{4}$/.test(regNumber.substring(6, 10))) {
            throw new Error("Last 4 characters must be digits.");
        }

        result.innerHTML = "Valid Registration Number";
        result.className = "valid";

    } catch (error) {

        result.innerHTML = "Invalid: " + error.message;
        result.className = "invalid";

    }
}

```

The Vehicle Registration Validator verifies whether the entered registration number follows the standard Indian vehicle registration format.

Features:

- Registration Number Input

- Automatic Validation
- Format Checking
- Exception Handling using Try-Catch
- Displays Success or Error Messages
- User-friendly Interface

The validator checks:

- Registration number is not empty
- Exactly 10 characters
- First two characters are uppercase alphabets
- Next two characters are digits
- Following two characters are uppercase alphabets
- Last four characters are digits

The application uses JavaScript validation functions together with try-catch blocks to handle invalid input and display meaningful error messages.

Features Implemented

- Function Declaration
- Function Expression
- Arrow Function
- Variable Scope
- Closures
- Try-Catch Exception Handling
- String Manipulation
- DOM Manipulation
- Input Validation
- Dynamic Result Display

Output

Task 4.a — Palindrome Checker

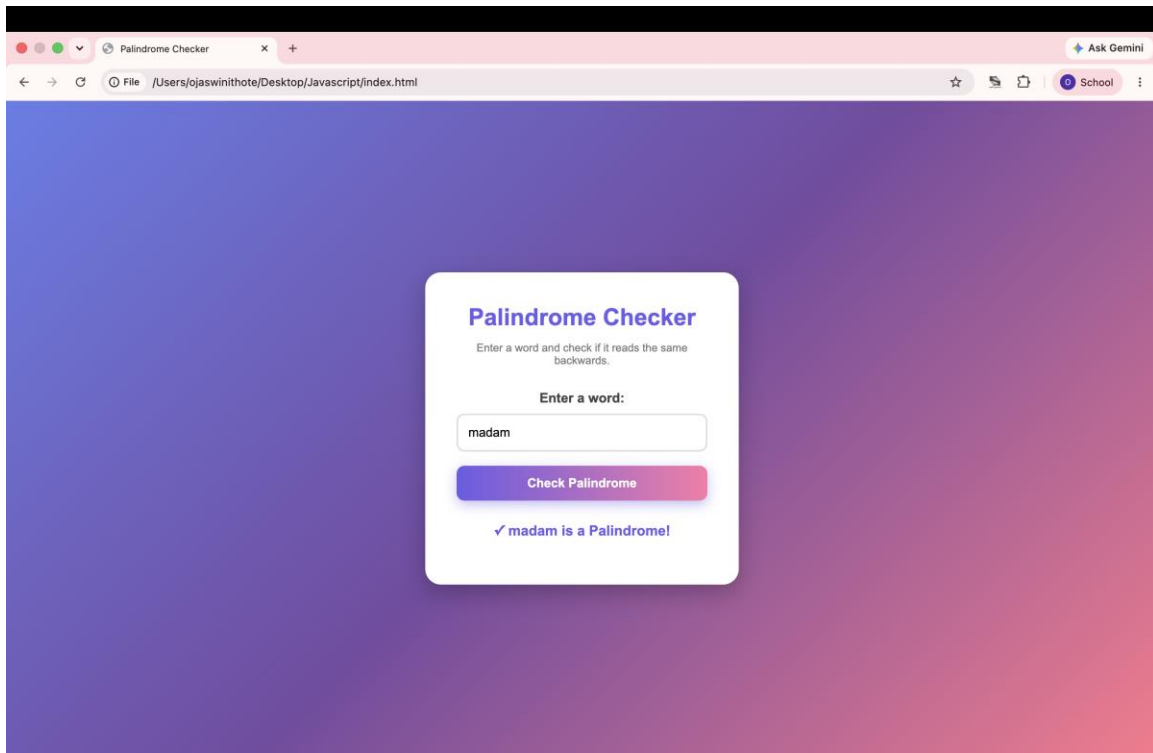
- User enters a word or phrase.
- Clicking **Check Palindrome** verifies whether the input is a palindrome.
- Appropriate success or failure message is displayed.
- Empty input is handled using a try-catch block.

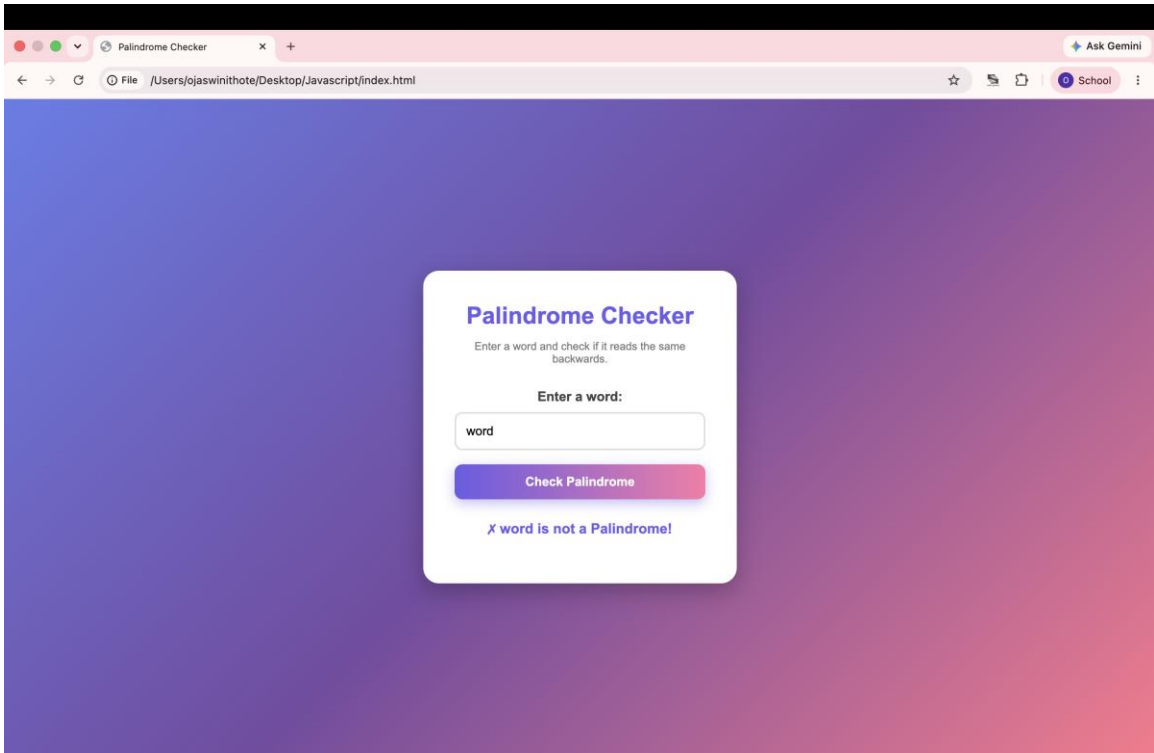
Task 4.b — Vehicle Registration Validator

- User enters a vehicle registration number.
- Clicking **Validate Registration** checks the format.
- Valid registration numbers display a success message.

- Invalid inputs display descriptive error messages generated through exception handling.

Screenshot





Case Study Title

Palindrome Checker and Vehicle Registration Validator using JavaScript Functions, Scope, Closures and Try-Catch Exception Handling

Case Study Program Code

The practical demonstrates different types of JavaScript functions and exception handling.

Concept	Usage
Function Declaration	Main processing functions such as palindrome checking and registration validation
Function Expression	Helper functions for string manipulation and validation
Arrow Function	Displaying results and simplified function syntax
Scope	Local and global variables used during validation
Closures	Counter function demonstrating preserved variables
Try-Catch	Handles invalid or empty user input gracefully
String Manipulation	Reversing strings and validating registration numbers
DOM Manipulation	Dynamically displaying validation results

Output Summary

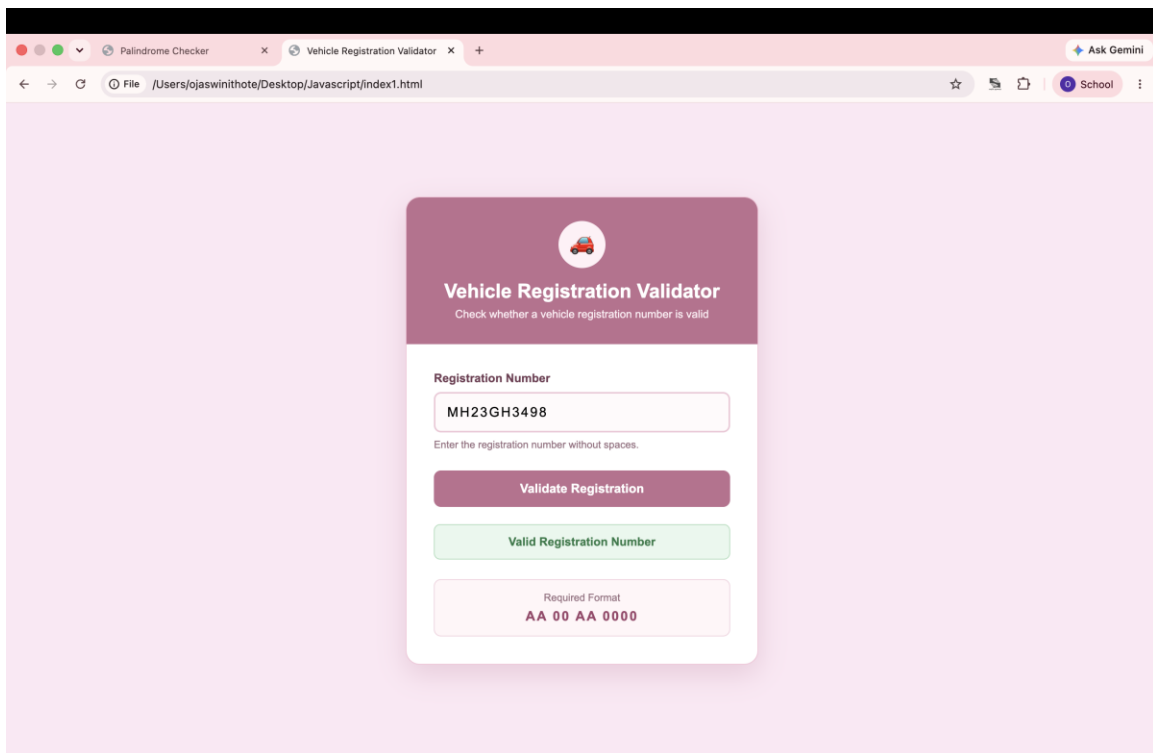
Task 4.a

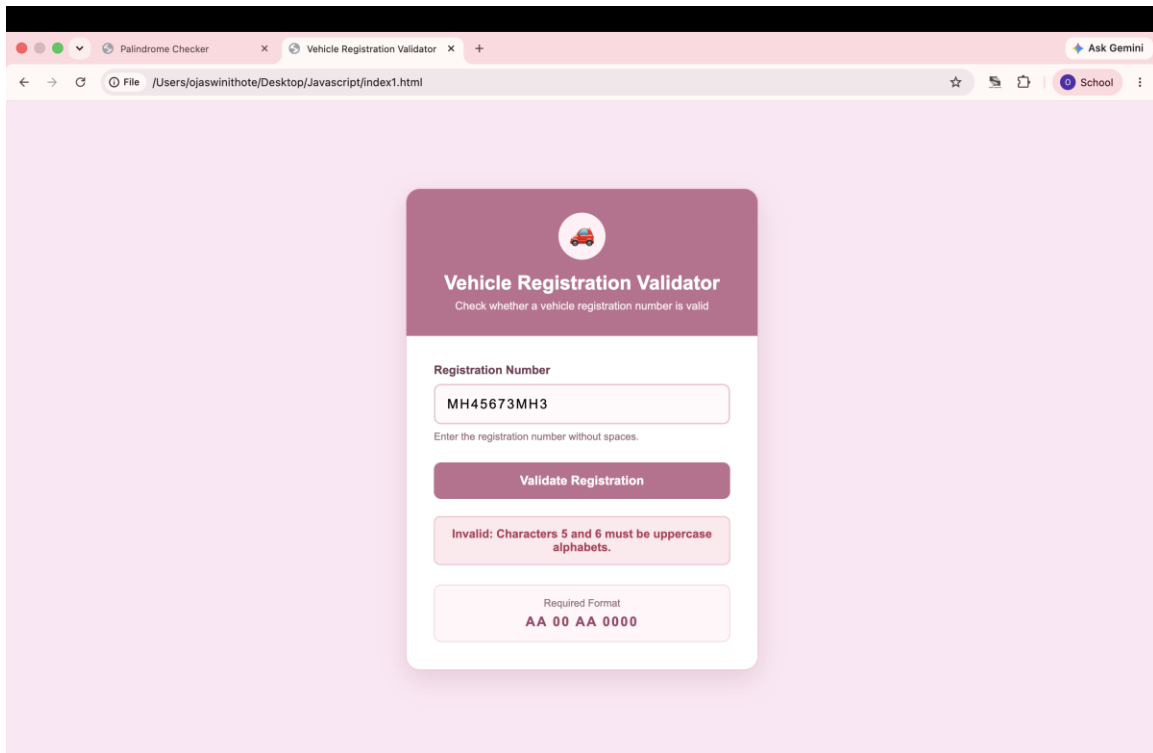
- Accepts user input.
- Removes unnecessary characters.
- Determines whether the input is a palindrome.
- Displays the result dynamically.

Task 4.b

- Accepts a vehicle registration number.
- Validates each part of the registration format.
- Displays success or descriptive error messages.
- Prevents invalid registrations using exception handling.

Screenshots





Result / Conclusion

The experiment was completed successfully. Different JavaScript function types including **Function Declaration**, **Function Expression**, and **Arrow Functions** were implemented along with **variable scope**, **closures**, and **try-catch exception handling**. A **Palindrome Checker** and a **Vehicle Registration Validator** were developed to demonstrate string manipulation, input validation, DOM manipulation, and effective error handling techniques in JavaScript.